



K R I T I



M I D P R E P

 **SpeakX**
Project Aurora

K R I T I



KRITI

MID PREP

Introduction

Across EdTech, Consumer Apps, FinTech, HealthTech, and SaaS—user communication is the single most powerful, yet most poorly engineered, growth lever. Today's notification systems are rule-based, segment-blind, timing-agnostic, and completely unaware of what actually works for whom, when, and why.

By 2030, intelligent communication orchestration will be the competitive moat—knowing WHO to talk to, WHAT to say, WHEN to say it, WHY it works, and HOW to improve continuously. Your challenge is to build that system: a self-learning notification orchestrator that can extend to any B2C or B2B application.

EdTech (SpeakX) is the reference domain, but your architecture must be domain-agnostic—swap the Knowledge Bank and user data, keep the same orchestrator core.

About Us

SpeakX is an AI-powered English speaking and communication learning app that helps Hindi and regional language speakers in India transform from "I understand English but can't speak" to confident English speakers through daily, guided speaking practice with our AI teacher "Sia".

We target pan-India across Tier 1, 2, and 3 cities, with Tier 2 and Tier 3 being our primary audience. Our core users are working professionals, college students, and ambitious homemakers in the 20-45 age group seeking better jobs, promotions, and social confidence.

With features like AI tutor conversations, gamification (coins, streaks, leaderboards), progress reports, and role-based practice scenarios, we drive daily engagement and habit formation.



KRITI

MID PREP

Key metrics include Trial→Monthly conversion, Monthly Retention (Users who complete an exercise within the Month out of all users who converted from trial to monthly user), W1 Retention (Users who complete an exercise within the 1st week after trial to monthly conversion out of all users who converted from trial to monthly user) across different user personas. These numbers represent significant opportunities for improvement through intelligent, personalized communication.

Problem Statement Description

Design and build a fully working, self-learning notification and communication orchestrator that demonstrates how user engagement can become smarter, more personalized, and continuously improving. Your solution should combine data intelligence, AI-powered content generation, and behavioral learning across three integrated tasks:

Task 1: System Architecture & Intelligence Design

Build the core intelligence engines that power the orchestrator:

- Knowledge Bank Engine: Ingest company KB (text/markdown/PDF), auto-infer North Star metric, extract feature-goal mappings, define allowed tones and Octolysis-inspired hooks
- User Data Ingestion: Accept behavioral CSV data (demographics, psychographics, feature usage, temporal patterns), validate schema, handle missing data, create user profiles
- MECE Segmentation Engine: Generate 6-12 mutually exclusive segments with propensity scores (gamification, AI tutor, leaderboard, social), activeness scores, and churn risk signals
- Goal & Journey Builder: Define primary goals, sub-goals, and day-on-day progression for each segment x lifecycle stage (Trial D0-D7, Paid D8-D30, Churned/Inactive)



Task 2: Communication & Timing Intelligence

Build the messaging and timing optimization layers:

- Theme Engine: Define most relevant communication themes per segment using Octolysis 8 Core Drives (Epic Meaning, Accomplishment, Empowerment, Ownership, Social Influence, Scarcity, Unpredictability, Loss Avoidance)
- Template Generator: Create exactly 5 message templates per Segment x Lifecycle x Goal x Theme combination, with bilingual content (Hindi + English), tone, hook, and feature reference
- Timing Optimizer: Define 6 standard time windows (early_morning 06-09, mid_morning 09-12, afternoon 12-15, late_afternoon 15-18, evening 18-21, night 21-24), learn optimal windows per segment
- Frequency Optimizer: Determine 3-9 notifications/day based on segment activeness ($>0.7 = 7-9/\text{day}$, $0.4-0.7 = 5-6/\text{day}$, $<0.4 = 3-4/\text{day}$) with guardrail override (uninstall $>2\% = \text{reduce by } 2$)

Task 3: Execution & Self-Learning

Build the output generation and learning feedback loop:

- Schedule Generator: Output user-wise notification schedule (user_id, segment, lifecycle_day, notif_1 through notif_9 with template_id, time, channel) showing gradual journey progression
- Classification: When performance data (On Demo day, experiment_results provided by SpeakX) is uploaded, classify templates as GOOD (CTR $>15\%$, Engagement $>40\%$), NEUTRAL (5-15%, 20-40%), or BAD ($<5\%$, $<20\%$)
- Learning Engine: Suppress BAD templates, promote GOOD as references, learn timing patterns per segment, generate improved Iteration 1 outputs with measurable delta
- Delta Reporter: Document every change (entity_type, entity_id, change_type, metric_trigger, before_value, after_value, explanation) with causal reasoning



Task 2: Communication & Timing Intelligence

Build the messaging and timing optimization layers:

- Theme Engine: Define most relevant communication themes per segment using Octolysis 8 Core Drives (Epic Meaning, Accomplishment, Empowerment, Ownership, Social Influence, Scarcity, Unpredictability, Loss Avoidance)
- Template Generator: Create exactly 5 message templates per Segment x Lifecycle x Goal x Theme combination, with bilingual content (Hindi + English), tone, hook, and feature reference
- Timing Optimizer: Define 6 standard time windows (early_morning 06-09, mid_morning 09-12, afternoon 12-15, late_afternoon 15-18, evening 18-21, night 21-24), learn optimal windows per segment
- Frequency Optimizer: Determine 3-9 notifications/day based on segment activeness ($>0.7 = 7-9/\text{day}$, $0.4-0.7 = 5-6/\text{day}$, $<0.4 = 3-4/\text{day}$) with guardrail override (uninstall $>2\% = \text{reduce by } 2$)

Task 3: Execution & Self-Learning

Build the output generation and learning feedback loop:

- Schedule Generator: Output user-wise notification schedule (user_id, segment, lifecycle_day, notif_1 through notif_9 with template_id, time, channel) showing gradual journey progression
- Classification: When performance data (On Demo day, experiment_results provided by SpeakX) is uploaded, classify templates as GOOD (CTR $>15\%$, Engagement $>40\%$), NEUTRAL (5-15%, 20-40%), or BAD ($<5\%$, $<20\%$)
- Learning Engine: Suppress BAD templates, promote GOOD as references, learn timing patterns per segment, generate improved Iteration 1 outputs with measurable delta
- Delta Reporter: Document every change (entity_type, entity_id, change_type, metric_trigger, before_value, after_value, explanation) with causal reasoning



Input Data Schemas

Column	Type	Description
user_id	string	Unique anonymized user identifier
lifecycle_stage	enum	trial paid churned inactive
days_since_signup	int	Days since registration
age_band, region	string	Demographics (18-24/25-34/35-44/45+, tier1/tier2/tier3)
sessions_last_7d, exercises_completed_7d	int	Behavioral activity counts
streak_current, coins_balance	int	Gamification state
feature_ai_tutor_used, feature_leaderboard_viewed,	bool	Feature usage flags
preferred_hour	int	Most active hour (0-23)
notif_open_rate_30d, motivation_score	float	Engagement metrics (0-1)

Octolysis 8 Core Drives Reference

#	Core Drive	Example Message Hook
1	Epic Meaning	"Join 1M+ learners transforming their careers"
2	Accomplishment	"You completed 5 lessons! Keep the momentum"
3	Empowerment	"Choose your learning path today"
4	Ownership	"Your 50 coins are waiting to be spent"
5	Social Influence	"3 friends joined this week. Invite more!"
6	Scarcity	"Only 2 days left in your trial!"
7	Unpredictability	"Surprise reward inside today's lesson!"
8	Loss Avoidance	"Your 7-day streak is at risk!"



Experiment Results CSV Schema (Provided for Demo2 by SpeakX)

Column	Type	Description
template_id	string	Unique template identifier
segment_id	string	Target segment ID
lifecycle_stage	string	User lifecycle stage
goal	string	Template goal alignment
theme	string	Octalysis hook used
notification_window	string	Time window of delivery
total_sends	integer	Number of notifications sent
total_opens	integer	Number of notifications opened
total_engagements	integer	Number of in-app actions taken
ctr	float	Click-through rate (opens / sends)
engagement_rate	float	Engagement rate (engagements / opens)
uninstall_rate	float	Uninstall rate post-notification
performance_status	string	Template classification (GOOD/BAD/NEUTRAL)

Default Thresholds & Configuration

Template Classification Cutoffs

Classification	CTR	Engagement Rate	Action
Good	> 15%	> 40%	Promote, use as reference
Neutral	5-15%	20-40%	Iterate, A/B test further
Bad	< 5%	< 20%	Suppress, do not use



Notification Frequency Bands

Activeness Score	Notifs/day	Churn Risk	Strategy
> 0.7 (Highly Active)	7-9	Low	Maximum engagement
0.4-0.7 (Moderate)	5-6	Medium	Balanced approach
< 0.4 (Low Active)	3-4	High	Conservative, avoid fatigue

GUARDRAIL OVERRIDE: If segment `uninstall_rate` > 2%, reduce notification frequency by 2/day regardless of activeness score.

Standard Time Windows

Window Name	Time Range	Typical Use
early_morning	06:00 - 08:59	Morning motivation, habit trigger
mid_morning	09:00 - 11:59	Work break reminder
afternoon	12:00 - 14:59	Lunch break engagement
late_afternoon	15:00 - 17:59	Productivity boost
evening	18:00 - 20:59	Post-work learning
night	21:00 - 23:59	End-of-day recap, streak save



Deliverables

1. Task 1: System Architecture & Intelligence

- `company_north_star.json` - Inferred North Star, justification, measurable proxy
- `feature_goal_map.json` - Feature → lifecycle → outcome mapping
- `allowed_tone_hook_matrix.json` - Allowed/disallowed tones, hook taxonomy
- `user_segments.csv` - MECE segments with propensities, activeness, churn risk
- `segment_goals.csv` - Goals, sub-goals, day-on-day focus per segment x lifecycle

2. Task 2: Communication & Timing Intelligence

- `communication_themes.csv` - Primary/secondary themes, tone preferences, hooks per segment
- `message_templates.csv` - 5 templates per combination with bilingual content (Hindi + English)
- `timing_recommendations.csv` - Optimal windows per segment with expected CTR/engagement
- `user_notification_schedule.csv` - User-wise daily schedule (3-9 notifs with template, time, channel)

3. Task 3: Execution & Self-Learning

- `experiment_results.csv` - (Provided by SpeakX) Performance data with CTR, engagement per template/timing
- `learning_delta_report.csv` - Changes between iterations with causal explanations
- Demo showing both iterations (before/after learning) in action
- Complete downloadable codebase (Private GitHub repository)
- `README.txt` (≤500 words) detailing architecture, models, and run instructions



KRITI

MID PREP

Live Demo Flow

Phase 1: Initial Run (Iteration 0 - Before Learning)

- Evaluator provides Knowledge Bank (company KB text/markdown)
- Evaluator provides User Behavioral Dataset (CSV per schema)
- System processes inputs and generates ALL Iteration 0 outputs
- Contestant exports files to iteration_0_before_learning/ folder
- Evaluator reviews: segmentation, templates, timing, frequency, schedules

Phase 2: Learning Run (Iteration 1 - After Learning)

- Evaluator classifies templates as GOOD/NEUTRAL/BAD per cutoffs
- Evaluator provides experiment_results.csv (CTR, engagement per template/timing)
- System ingests performance data
- System relearns and generates improved Iteration 1 outputs
- Contestant exports files to iteration_1_after_learning/ folder
- Evaluator compares Iteration 0 vs 1 for MEASURABLE improvement

Non-Negotiable Constraints

- Fully working end-to-end system with complete downloadable codebase
- Runnable locally for live demos (to catch/prevent hacky workarounds)
- Accept NEW datasets at runtime (we provide data on-the-spot during finals)
- Demonstrate REAL learning (Iteration 0 → Iteration 1 with measurable delta)
- Export ALL outputs in strict CSV/JSON formats per schemas defined
- Be explainable, auditable, and reproducible

AUTO-FAIL: Hardcoded outputs • Mock learning • PPTs without exports • No delta between iterations • Missing/renamed files • Merged iterations



Submission Structure

```
SpeakX_AURORA_<TeamName>.zip
├─ iteration_0_before_learning/
│   ├── company_north_star.json
│   ├── feature_goal_map.json
│   ├── allowed_tone_hook_matrix.json
│   ├── user_segments.csv
│   ├── segment_goals.csv
│   ├── communication_themes.csv
│   ├── message_templates.csv
│   ├── timing_recommendations.csv
│   └─ user_notification_schedule.csv
├─ iteration_1_after_learning/
│   ├── user_segments.csv (if changed)
│   ├── message_templates.csv (updated)
│   ├── timing_recommendations.csv (updated)
│   └─ user_notification_schedule.csv (updated)
├─ experiment_results.csv
├─ learning_delta_report.csv
├─ codebase/ (complete runnable code)
└─ README.txt (≤500 words)
```

Evaluation Criteria

Dimension	Weight	Description
System Completeness	15%	All files present, correct schemas, runnable codebase, proper structure
Segmentation Quality	15%	MECE compliance, meaningful descriptions, differentiated propensities
Messaging Intelligence	25%	Template variety, theme relevance, bilingual quality, hook diversity, gradual journey
Timing & Frequency	10%	Segment-aware frequency (3-9/day), timing optimization, guardrail respect
Learning & Evolution	25%	Observable delta, causal explanations, real improvement in Iteration 1
Extensibility	5%	Domain-agnostic architecture, configurable for other industries
Presentation	5%	Overall clarity, professionalism, demo quality, README documentation